

吳承儒 Jack Wu

後端工程師 / Backend Engineer

hotrabbit1016@gmail.com | +886 955 684 107 | hotrabbit1016.github.io/resume

專業摘要

5 年後端開發經驗，專注於線上娛樂平台。主力為 C#/.NET，並使用 Go 建置稽核資料管道、以 Python 開發實體賽道控制與 AI 影像判定。工作橫跨交易與即時推播、分散式協調、硬體控制和電腦視覺。習慣優先處理失效路徑，包括重複或遺失的訊號、延遲、競態條件、AI 誤判與設備故障。

工作經歷

泰鑫軟體 | 後端工程師 | 2024/01 - 現在

- 原團隊與產品線在經營者更換後一同移轉；負責桌台客戶端重構、營運後台、聊天平台操作稽核，以及實體賽道控制與 AI 賽果判定。
- 以 .NET、Go 與 Python 開發正式環境服務，涵蓋多 Pod 協調、非同步稽核資料管道、硬體抽象層與即時影像推論。
- 約一年跨國遠端帶領一名菲律賓工程師，負責需求拆解、技術方向討論、程式碼審查與開發進度追蹤。

凱駁國際、鼎順數位科技 | 後端工程師 | 2021/04 - 2023/12

- 兩間公司由同一經營者與團隊運作，工作連續未中斷；從桌台報表轉檔與狀態處理，逐步負責玩家下注、取消下注與結算服務。
- 以交易流程確保下注一致性，結算完成後透過 SignalR 即時推播；跨服務狀態變更使用 RabbitMQ 事件處理。正式服務前後維護約四年。
- 開發現場 WPF 桌台操作端；其中一代主畫面與核心邏輯九成以上由我實作，並加入二次確認、錯誤分層與完整狀態記錄。

技術能力

後端: C# / .NET 10、ASP.NET Core、EF Core、Go (Gin、Ent、Uber Fx)、Python

即時與訊息: SignalR、WebSocket、RabbitMQ、Redis Streams

資料儲存: MySQL、Redis (Lua、分散式鎖)、ClickHouse、Elasticsearch

部署與維運: Docker、Kubernetes、Helm、GitLab CI、ArgoCD、OpenTelemetry

AI 與影像: YOLO、Roboflow、PyTorch、TensorFlow、OpenCV、ONNX

硬體介接: SPI (MCP23S17)、UART / RF、Raspberry Pi、基礎焊接與電路板設計討論

學歷與語言

國立中興大學 電機工程學系 | 中文 (母語) | 英文 (TOEIC 770)

桌台客戶端重構 | 2025 - 2026

.NET 10 | Redis | Kubernetes | xUnit

- 將只能在單機執行的 WPF 程式重構為可多 Pod 部署的服務；主程式與測試由我獨立設計實作。
- 使用 Redis + Lua 實作桌台租約，原子化 acquire、renew、release，避免兩個 Pod 同時接管同一張桌而重複處理訊號。
- 採 Supervisor-Worker、一桌一個 DI scope；續租失敗立即停止 worker。以 State Pattern 與 TimeProvider 測試八種桌台狀態和調度核心。
- 移除版控中的明文密碼與連線字串，改由 Kubernetes Secret 在部署時注入。

聊天平台操作稽核 | 2026

Go | Gin | Redis Streams | ClickHouse | Uber Fx

- 從方案評估、設計文件到上線獨立完成，讓稽核與客服可查詢誰在何時修改了什麼。
- 主請求將紀錄送入 Redis Stream，由 Consumer Group 非同步寫入 ClickHouse；避免稽核拖慢操作，並以分區和 TTL 管理資料。
- 分別在 Middleware 與 Service 蒐集 HTTP 語意及前後狀態，最後匯流；將 ID 與 enum 轉成稽核人員可直接理解的名稱。
- 將查詢條件做成封閉型別並共用驗證邏輯，無效值直接回 400；時區界線保留寬鬆後端防線，精準限制由前端負責。

實體賽道控制與韌體 | 2024 - 2026

Python | SPI | UART / RF | Raspberry Pi | Flask | OpenTelemetry

- 獨立開發 8 條實體彈珠賽道的控制軟體與監控介面，涵蓋感測器、閘門、賽事流程與 OBS 直播畫面切換。
- 透過 SPI 驅動 MCP23S17，讀取最多 28 路光學感測器，並自動辨識兩代感測板；無硬體時可退回模擬模式。
- 針對會遺失的 RF 訊號設計重試、重新初始化、連線檢查與資源釋放等多層復原機制，使設備異常後可自行恢復。
- 以 Strategy + Factory 隔離八條賽道的不同流程，三支程式由 systemd 分別常駐，監控介面失效不會中斷賽道控制。

彈珠賽事影像判定 | 2024 - 2026

Python | YOLO | Roboflow | PyTorch | OpenCV | ONNX | FastAPI

- 獨立完成模型訓練、判定邏輯、查詢 API 與現場介面；系統已上線並取代人工影像判讀。
- 把每顆球訓練成獨立 class，避開多物件追蹤的 ID switch；以四狀態機處理落球判定。
- 所有門檻使用時間而非 frame 數，透過二段確認、存活門檻與 stale 清理分別吸收漏偵測、誤判和 ghost track。
- 排名採最後偵測時間而非確認落球時間，避免確認延遲影響公平性；推論放在獨立 thread，保持現場介面可操作。